The University of Nottingham
Malaysia Campus

SCHOOL OF COMPUTER SCIENCE

A LEVEL 2 MODULE, SPRING SEMESTER 2023-24

ALGORITHM CORRECTNESS AND EFFICIENCY (COMP 2038)

Time allowed TWO hours

Candidates may complete the front cover of their answer book and sign their desk card but must NOT write anything else until the start of the examination period is announced

Answer ALL questions

This module has 25% coursework assessment and 75% examination.

All questions and parts of questions carry remaining marks as indicated in brackets.

Only silent, self-contained calculators with a Single-Line Display or Dual-Line Display are permitted in this examination.

Dictionaries are not allowed with one exception. Those whose first language is not English may use a standard translation dictionary to translate between that language and English provided that neither language is the subject of this examination. Subject specific translation dictionaries are not permitted.

No electronic devices capable of storing and retrieving text, including electronic dictionaries, may be used.

DO NOT turn examination paper over until instructed to do so

ADDITIONAL MATERIAL: Answer booklets

INFORMATION FOR INVIGILATORS:

Questions papers and the answer booklets should be collected in at the end of the exam – do not allow candidates to take copies from the exam room.

Topic: Intuitionistic Logic and Classical Logic **[Total: 35 marks]**

1. Write Lean code to verify the following theorem.

(a)

```
variables P Q : Prop

theorem test1 : P → Q → P ∧ Q ∧ P :=
begin
  sorry,
end
```

[6 marks]

(b)

```
variables P Q R : Prop

theorem test2 : (P → Q) ∨ (P → R) → P → (Q ∨ R) :=
begin
  sorry,
end
```

[9 marks]

2 (a) Write the two De Morgan's Laws in symbols.

[4 marks]

(b) Construct a truth table to verify both De Morgan's Law that you have written in **(a)**.

[10 marks]

(c) State in specific which of the two De Morgan's Law is not provable using intuitionistic logic.

[2 marks]

3 (a) Describe of the Law of Excluded Middle (EM).

[2 marks]

(b) Write the corresponding Lean statement to the Law of Excluded Middle (EM).

[2 marks]

Topic: Predicate Logic **[Total: 25 marks]**

4 Analyse the following statement.

If all human are superheroes, then if all superheroes have mental problems, then all human have mental problems.

(a) Translate the statement above into a proposition.

[6 marks]

(b) Write Lean code to verify the statement, with declaration of the variables used.

[6 marks]

5 (a) There are three properties that an equivalence relation possesses. One of them is transitivity.
Explain the other two properties with symbols.

[6 marks]

(b) Write Lean code to verify the transitivity property of a relation.

```
variables A : Type

theorem trans : ∀ x y z : A, x = y → y = z → x = z :=
begin
  sorry,
end
```

[7 marks]

Topic: Boolean **[Total: 25 marks]**

6 (a) Define ***and*** and ***or*** for Boolean using Lean code.

[10 marks]

(b) Explain why they are being defined in this way.

[8 marks]

7 Similar to Law of Excluded Middle (EM) in classical logic, every element in Boolean is either ***true*** `(tt)` or ***false*** `(ff)`.
Verify this statement through Lean.

[7 marks]

Topic: Natural Numbers, Lists and Trees **[Total: 15 marks]**

8 Analyse the following definition related to function `play`, that involves the natural number.

```
open nat

def play : nat → nat
| zero := 1
| (succ zero) := 0
| (succ (succ n)) := succ (succ (play n))
```

(a) State the result of `#reduce play 2` and `#reduce play 1`.

[2 marks]

(b) Explain your answer in **(a)**.

[3 marks]

9 (a) Explain briefly how insertion sort works.

[3 marks]

(b) Analyse the following Lean code.

```
open nat
open list

def ble : ℕ → ℕ → bool
| 0 n := tt
| (succ m) 0 := ff
| (succ m) (succ n) := ble m n

def ins : ℕ → list ℕ → list ℕ
| a [] := [a]
| a (b :: bs) := if ble a b then a :: b :: bs else b::(ins a bs)
```

Write down the result of `#reduce ins 4 [1, 3, 5, 7, 9]`.

[2 marks]

10 The following Lean code shows the definition of a binary tree.

```
inductive Tree : Type
| leaf : Tree
| node : Tree → ℕ → Tree → Tree
```

Explain both constructors stated in the definition above.

[5 marks]

Answer

1 (a)

```
variables P Q : Prop

theorem test1 : P → Q → P ∧ Q ∧ P :=
begin
    assume p q,
    constructor,
    exact p,
    constructor,
    exact q,
    exact p,
end
```

[application]

(b)

```
variables P Q R : Prop

theorem test2 : (P → Q) ∨ (P → R) → P → (Q ∨ R) :=
begin
    assume pqpr p,
    cases pqpr with pq pr,
    left,
    apply pq,
    exact p,
    right,
    apply pr,
    exact p,
end
```

[application]

2 (a) `¬(P ∨ Q) ↔ ¬P ∧ ¬Q` `¬(P ∧ Q) ↔ ¬P ∨ ¬Q`

[knowledge]

(b)

P	Q	P ∨ Q	¬(P ∨ Q)	¬P	¬Q	¬P ∧ ¬Q	P ∧ Q	¬(P ∧ Q)	¬P ∨ ¬Q
0	0	0	1	1	1	1	0	1	1
0	1	1	0	1	0	0	0	1	1
1	0	1	0	0	1	0	0	1	1
1	1	1	0	0	0	0	1	0	0

[application]

(c) `¬(P ∧ Q) → ¬P ∨ ¬Q`

[comprehension]

3 (a) Every proposition is either true or false.

[knowledge]

(b) `case em P with p np,`

[comprehension]

4 (a) `∀x : A, PP x → (∀y : A, PP y → QQ y) → ∀z : A, QQ z`
`A`: human `PP`: superheroes `QQ`: mental problems

[comprehension]

(b)

```
variables A : Type
variables PP QQ : A → Prop

example : (∀ x : A, PP x) → (∀ y : A, PP y → QQ y)
            → (∀ z : A, QQ z) :=
begin
  assume p pq a,
  apply pq,
  apply p,
end
```

[application]

5 (a) reflexive (∀ x : A, x = x), symmetric (∀ x y : A, x = y → y = x)

[knowledge]

(b)

```
variables A : Type

theorem trans1 : ∀ x y z : A, x = y → y = z → x = z :=
begin
  assume x y z xy yz,
  rewrite xy,
  exact yz,
end
```

[application]

6 (a)

```
def band : bool → bool → bool
| tt b := b
| ff b := ff

def bor : bool → bool → bool
| tt b := tt
| ff b := b
```

[comprehension]

(b) For and, if the first argument is true, then the truth value is depending on the second argument, which is b. In this case, tt : bool → bool is the identity on the second argument, because true and true = true and true and false = false. If the first argument is false, then the outcome will be false regardless of what the second argument is.

For or, if the first argument is true, then the outcome will always be true regardless of what the second argument is. If the first argument is false, then the truth value is depending on the second argument, because false or true = true and false or false = false. In this case, bor ff : bool → bool is the identity on the second argument.

[comprehension]

7
```
example : ∀ x : bool, x = tt ∨ x = ff :=
begin
  assume x,
  cases x,
  right,
  refl,
  left,
  refl,
end
```
[application]

8 (a) 3 and 0

[comprehension]

(b)
```
play 2= play (succ (succ zero))
      = succ (succ (play zero))
      = succ (succ (1))
      = 3

play 1= play (succ zero)
      = 0
```
[comprehension]

9 (a) Start with the second element (index 1) of the array.
Compare the second element with the one before it. If the second element is smaller, swap them.
Move to the next element (index 2) and compare it with the elements before it until finding its correct position.
Repeat this process until the entire array is sorted.

[knowledge]

(b) [1, 3, 4, 5, 7, 9]

[comprehension]

10
```
leaf : Tree
```

This is the base case of the inductive definition. It represents a tree with no children, which is a leaf node with no further branches. This is the simplest form of a tree.

```
node : Tree → ℕ → Tree → Tree
```

This specifies that to construct a new tree (a node), three components are needed:

1. A left subtree, denoted by Tree. This can be either another tree or one of the base cases (leaf).
2. A natural number (ℕ), which represents the value stored in the node.
3. A right subtree, denoted by Tree.

[comprehension]

END